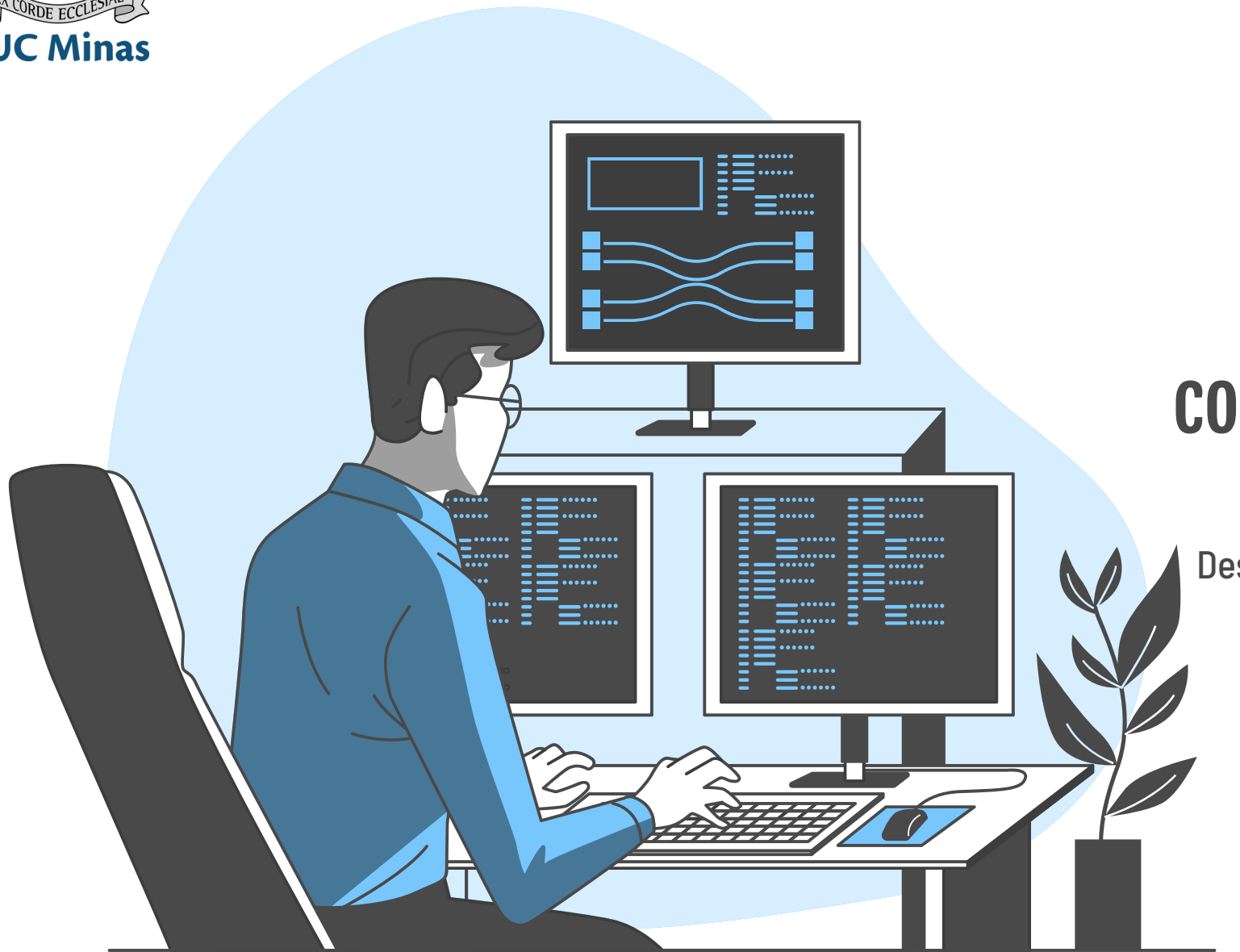
PUC Minas



Aula 05 · Back-end

API REST com Spring: controllers, status e erros

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Mapear os verbos **GET, POST, PUT e DELETE** para um CRUD

2

Usar `@PathVariable`, `@RequestParam` e `@RequestBody`

3

Responder com o **status HTTP correto** usando `ResponseEntity`

4

Validar entradas com `@Valid` e tratar erros com `@RestControllerAdvice`

5

Testar a API com cliente HTTP e `curl`

Antes de começar

- Aula 04: camadas e pacotes

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 REST em uma página

02 O controller completo

03 Validando a entrada

04 Tratamento global de erros

05 Testando a API

06 Aprofundando: APIs REST profissionais

01

REST em uma página

REST em uma página

Uma API **REST** organiza tudo em **recursos** (alunos, livros, produtos), cada um com uma URL, e usa os verbos do HTTP para dizer o que fazer:

Verbo	URL	Ação	Sucesso
`GET`	`/api/alunos`	Listar	`200 OK`
`GET`	`/api/alunos/{id}`	Buscar um	`200 OK`
`POST`	`/api/alunos`	Criar	`201 Created`
`PUT`	`/api/alunos/{id}`	Atualizar	`200 OK`
`DELETE`	`/api/alunos/{id}`	Excluir	`204 No Content`

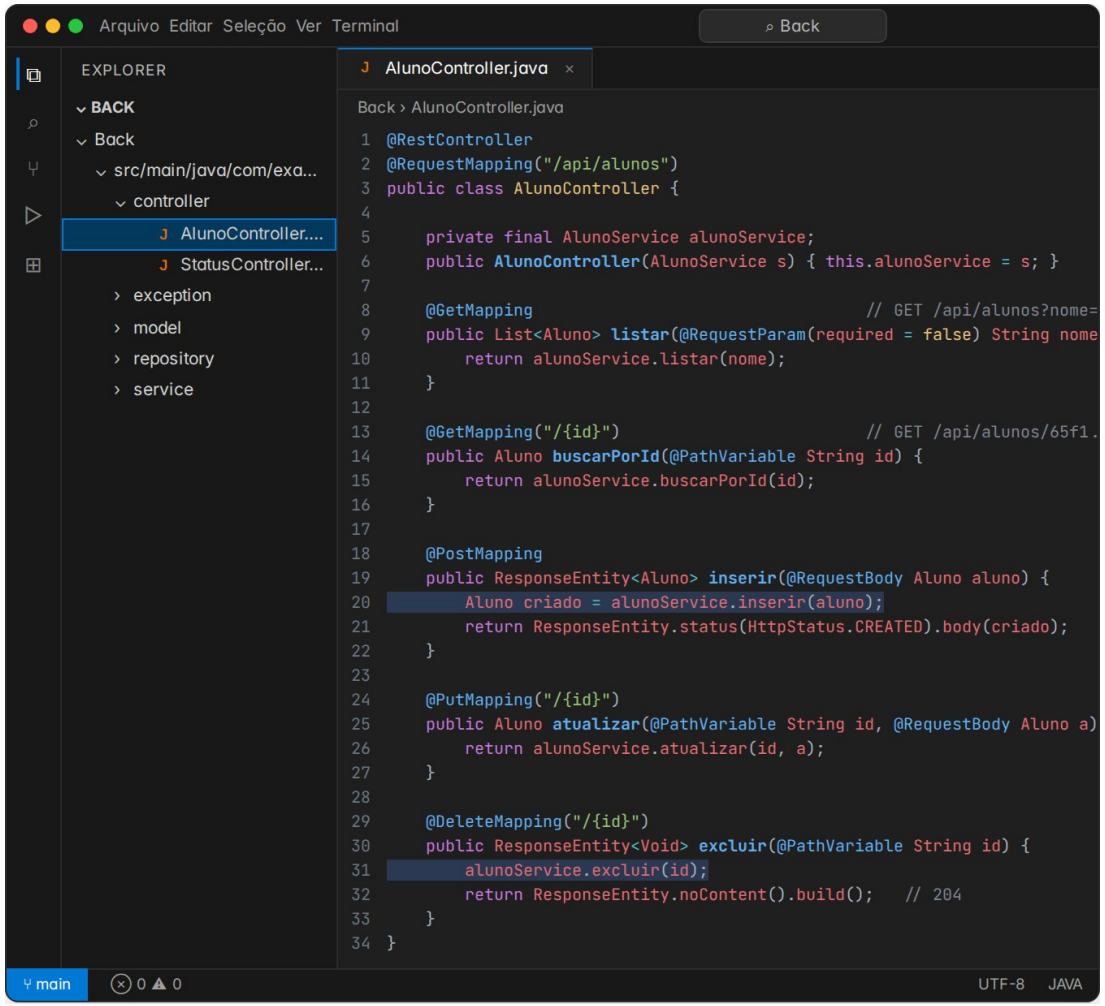
Códigos de status que você precisa conhecer

Faixa	Significado	Exemplos
2xx	Deu certo	`200 OK`, `201 Created`, `204 No Content`
4xx	Erro do cliente	`400 Bad Request` (dados inválidos), `401 Unauthorized` (sem login), `403 Forbidden` (sem permissão), `404 Not Found`, `405 Method Not Allowed`
5xx	Erro do servidor	`500 Internal Server Error`, `503 Service Unavailable` (banco fora do ar)

02

O controller completo

O controller completo



`AlunoController` do projeto `Exemplo_Integracao`: um CRUD completo em cinco métodos.

O controller completo

Anotação	De onde vem o valor	Exemplo
`@PathVariable`	Pedaço da URL	`/api/alunos/{id}` → `id`
`@RequestParam`	*Query string* depois do `?`	`/api/alunos?nome=ana` → `nome`
`@RequestBody`	Corpo JSON da requisição	`{ "nome": "Ana" }` → objeto `Aluno`
`@RequestHeader`	Cabeçalho HTTP	`Authorization: Bearer ...`

O controller completo

O Spring converte o JSON do corpo em objeto Java (e o objeto de volta em JSON) automaticamente, usando a biblioteca **Jackson**.

02

O controller completo

Aula 05 · Back-end

O controller completo

Observação

No Spring Boot 4 o Jackson foi atualizado para a versão 3, com pacotes `tools.jackson.*` no lugar de `com.fasterxml.jackson.*`. Para você, que só devolve objetos no controller, nada muda. Só preste atenção se for criar um `ObjectMapper` à mão.

03

Validando a entrada

Validando a entrada

Com o starter `spring-boot-starter-validation`, anote o DTO e peça a validação com `@Valid`. Se falhar, o Spring lança `MethodArgumentNotValidException` antes mesmo de chegar no service.

Validação com Bean Validation

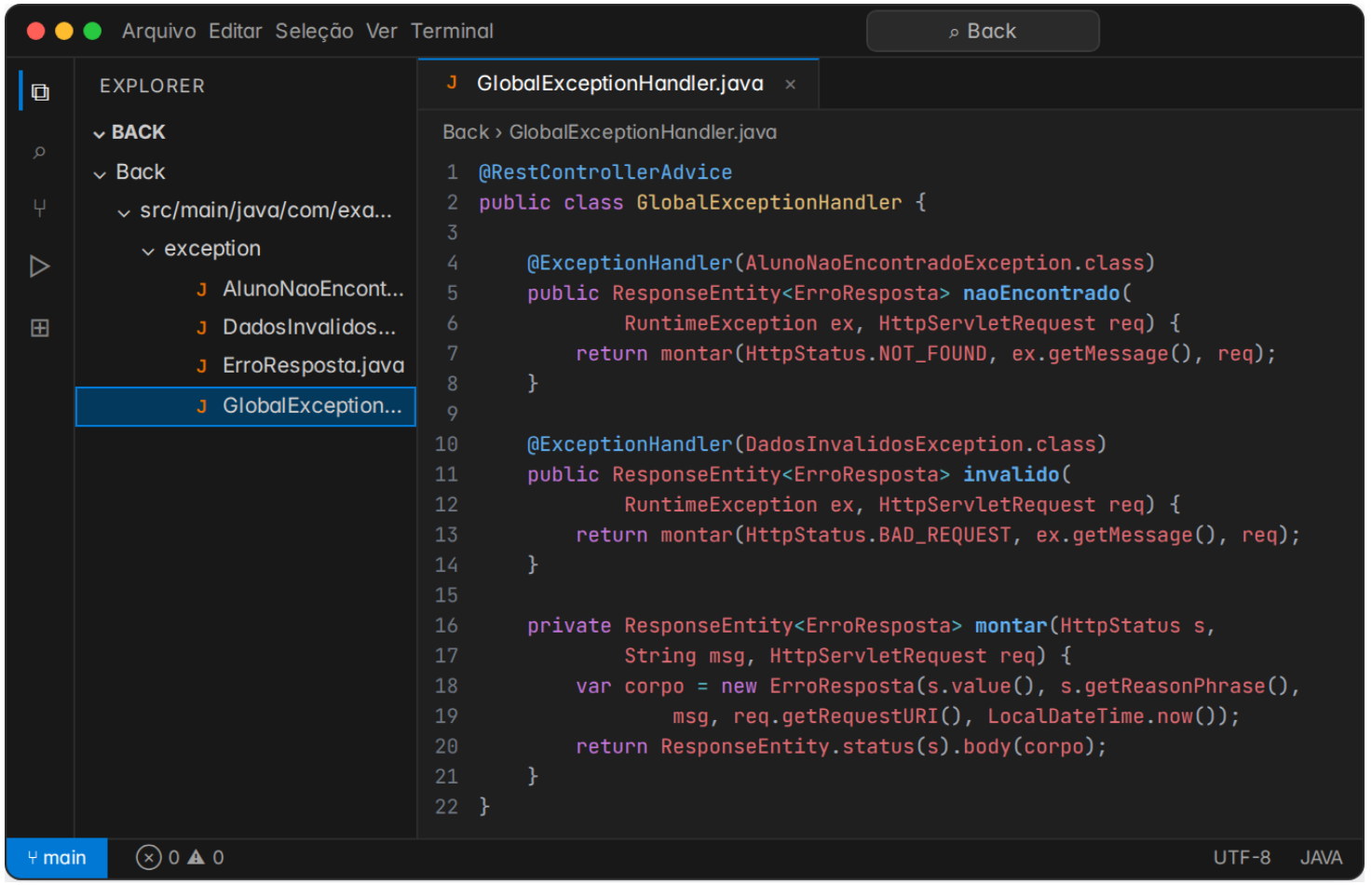
```
public record AlunoRequestDTO(
    @NotBlank(message = "O nome é obrigatório.") String nome,
    @NotBlank @Email(message = "E-mail inválido.") String email,
    @Size(max = 60) String curso) { }

@PostMapping
public ResponseEntity<AlunoResponseDTO> inserir(@Valid @RequestBody AlunoRequestDTO dto) { ... }
```

04

Tratamento global de erros

Tratamento global de erros



Em vez de `try/catch` em todos os métodos, o service **lança** exceções e uma única classe, anotada com `@RestControllerAdvice`, transforma cada uma em uma resposta JSON padronizada. O front-end React lê o campo `mensagem` e mostra na tela.

Tratador global: cada exceção vira um código HTTP e um JSON de erro.

Tratamento global de erros

exception/ (exceção própria e formato do erro)

```
public class AlunoNaoEncontradoException extends RuntimeException {
    public AlunoNaoEncontradoException(String id) {
        super("Aluno com id " + id + " não encontrado.");
    }
}

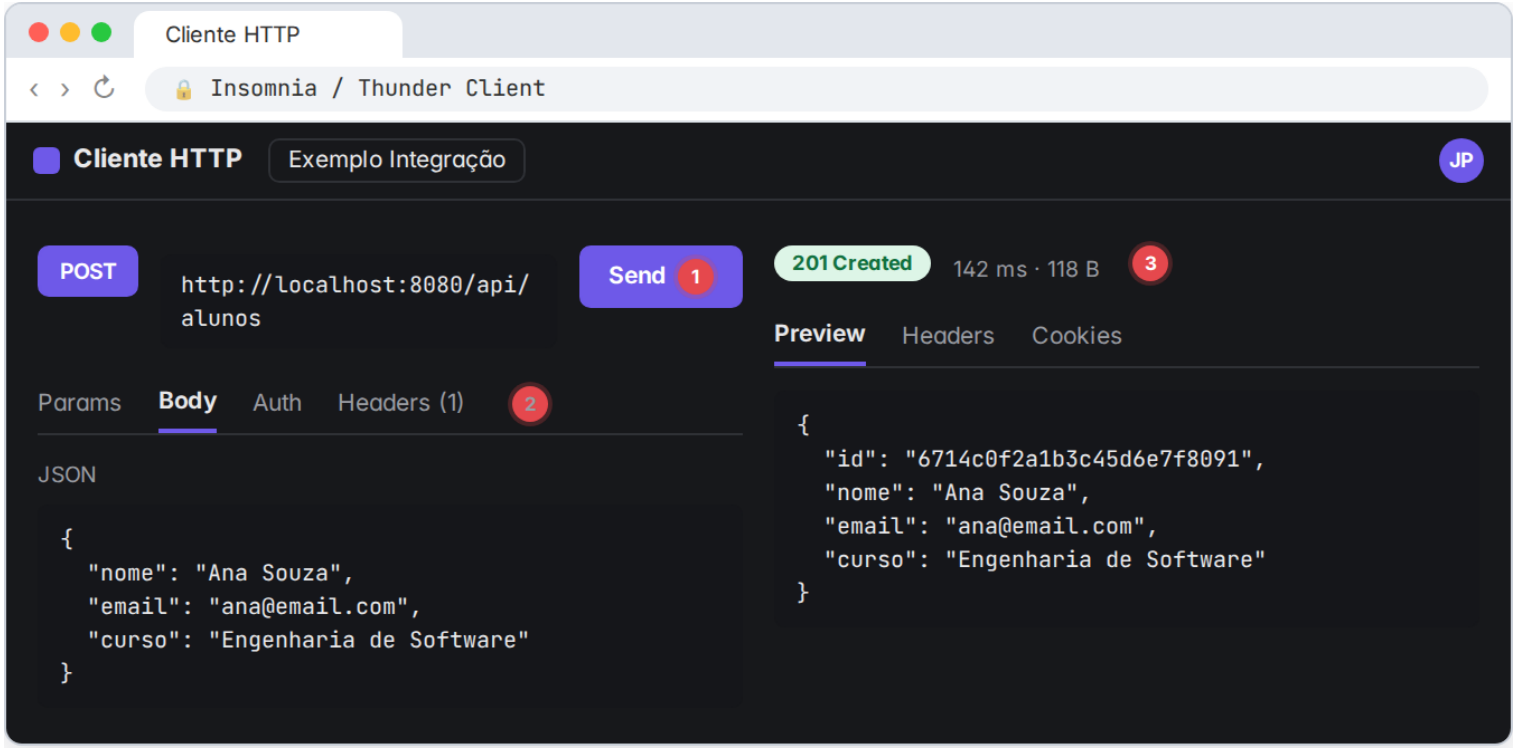
public record ErroResposta(int status, String erro, String mensagem,
                           String caminho, LocalDateTime timestamp) { }
```

05

Testando a API

Testando a API

O navegador só faz `GET` pela barra de endereços. Para testar `POST`, `PUT` e `DELETE`, use um **cliente HTTP**: Insomnia, Postman, Bruno ou a extensão **Thunder Client** / **REST Client** do VS Code.



Cliente HTTP enviando um `POST` com corpo JSON e recebendo `201 Created`.

Testando a API

1. Escolha o verbo (**POST**), cole a URL e clique em **Send**.
2. Na aba **Body**, escolha **JSON** e escreva o objeto. O cliente adiciona o cabeçalho `Content-Type: application/json`.
3. Confira o **status** e o corpo da resposta. Teste também os erros: envie sem `nome` e veja o `400`.

Testando a API

Os mesmos testes com curl

```
# listar
curl http://localhost:8080/api/alunos

# criar
curl -X POST http://localhost:8080/api/alunos \
     -H "Content-Type: application/json" \
     -d '{"nome":"Ana","email":"ana@email.com","curso":"ES"}'

# excluir (-i mostra o status 204)
curl -i -X DELETE http://localhost:8080/api/alunos/6714c0f2a1b3c45d6e7f8091
```

Testando a API

Dica

Os projetos da disciplina trazem uma pasta `requests/` com coleções exportadas do Insomnia. Importe o arquivo (`Insomnia_2026-03-28.yaml`, por exemplo) e todas as requisições já aparecem prontas.

06

Aprofundando: APIs REST profissionais

Aprofundando: APIs REST profissionais

Verbo	Seguro?	Idempotente?	Significado
GET	Sim	Sim	Só lê; repetir não muda nada
POST	Não	**Não**	Cada chamada cria um recurso novo
PUT	Não	Sim	Substitui o recurso inteiro; repetir dá o mesmo resultado
PATCH	Não	Depende	Altera só alguns campos
DELETE	Não	Sim	Apagar duas vezes deixa o mesmo estado (o segundo pode dar 404)

Aprofundando: APIs REST profissionais

Idempotente significa que repetir a mesma requisição produz o mesmo estado no servidor. Isso importa quando a rede falha e o cliente reenvia: reenviar um `PUT` é seguro; reenviar um `POST` pode duplicar o cadastro.

06

Aprofundando: APIs REST profissionais

Aula 05 · Back-end

Paginação com Spring Data

controller/AlunoController.java

```
@GetMapping
public Page<Aluno> listar(
        @PageableDefault(size = 10, sort = "nome") Pageable pageable) {
    return repo.findAll(pageable);
}

// GET /api/alunos?page=0&size=5&sort=nome,desc
// resposta: { "content": [...], "totalElements": 42, "totalPages": 9, "number": 0, ... }
```

201 Created com o cabeçalho Location

Boa prática REST

```
@PostMapping
public ResponseEntity<Aluno> inserir(@Valid @RequestBody AlunoRequestDTO dto,
                                     UriComponentsBuilder uri) {
    Aluno criado = service.inserir(dto);
    URI local = uri.path("/api/alunos/{id}").buildAndExpand(criado.getId()).toUri();
    return ResponseEntity.created(local).body(criado);   // Location: .../api/alunos/6714...
}
```

ProblemDetail: o formato padrão de erro

O Spring 6+ suporta o padrão **RFC 9457 (Problem Details)**, um JSON de erro reconhecido por várias ferramentas. Os projetos da disciplina usam um `ErroResposta` próprio, que é igualmente válido; o importante é ser **consistente** em toda a API.

Usando ProblemDetail

```
@ExceptionHandler(AlunoNaoEncontradoException.class)
public ProblemDetail naoEncontrado(AlunoNaoEncontradoException ex) {
    ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
    pd.setTitle("Aluno não encontrado");
    return pd;
}
// { "type": "about:blank", "title": "Aluno não encontrado", "status": 404,
//   "detail": "Aluno com id 99 não encontrado.", "instance": "/api/alunos/99" }
```

Aprofundando: APIs REST profissionais

Cabeçalho	Para que serve
`Content-Type: application/json`	Formato do corpo enviado
`Accept: application/json`	Formato que o cliente aceita receber
`Authorization: Bearer`	Credencial (JWT)
`Location`	URL do recurso recém-criado (201)
`Access-Control-Allow-Origin`	Origem autorizada pelo CORS

Boas práticas de nomes de rotas

Substantivos no plural

`/api/alunos`, nunca `/api/getAlunos`. O verbo é o método HTTP.

Hierarquia

`/api/cursos/3/alunos` para os alunos do curso 3.

Filtros na query string

`/api/alunos?curso=ES&ativo=true`

Versão

`/api/v1/alunos` quando a API precisa evoluir sem quebrar clientes antigos.

07

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual status representa sucesso em um `POST` que criou um recurso?

A 200 OK

B 201 Created

C 204 No Content

D 302 Found

Resposta

Qual status representa sucesso em um `POST` que criou um recurso?

A. 200 OK

✓ 201 Created

C. 204 No Content

D. 302 Found

Por quê: 201 indica criação; 204 é típico do DELETE.

Pergunta

De onde vem o valor de um parâmetro `@PathVariable`?

A. Do corpo JSON

B. De um pedaço da URL, como `/alunos/{id}`

C. Da query string

D. Do cabeçalho

Resposta

De onde vem o valor de um parâmetro `@PathVariable`?

A. Do corpo JSON

✓ De um pedaço da URL, como `/alunos/{id}`

C. Da query string

D. Do cabeçalho

Por quê: Query string é `@RequestParam`; corpo é `@RequestBody`.

Pergunta

Qual verbo não é idempotente?

A GET

B PUT

C DELETE

D POST

Resposta

Qual verbo não é idempotente?

A GET

B PUT

C DELETE

✓ POST

Por quê: Repetir um POST pode criar dois registros.

Quiz · questão 4 de 5

Pergunta

O que faz uma classe com `@RestControllerAdvice`?

A Cria endpoints

B Trata exceções de todos os controllers em um só lugar

C Conecta ao banco

D Gera tokens

Resposta

O que faz uma classe com `@RestControllerAdvice`?

A. Cria endpoints

✓ Trata exceções de todos os controllers em um só lugar

C. Conecta ao banco

D. Gera tokens

Por quê: É o GlobalExceptionHandler que converte exceções em status + JSON.

Quiz · questão 5 de 5

Pergunta

Por que testar `POST` e `DELETE` com cliente HTTP ou curl?

A O navegador só faz GET pela barra de endereços

B O navegador não entende JSON

C O Spring bloqueia o navegador

D Para evitar CORS no servidor

Resposta

Por que testar `POST` e `DELETE` com cliente HTTP ou curl?

✓ O navegador só faz GET pela barra de endereços

B O navegador não entende JSON

C O Spring bloqueia o navegador

D Para evitar CORS no servidor

Por quê: Para outros verbos é preciso uma ferramenta que monte a requisição.

Fechamento

Resumo da aula

- ✓ Recursos têm URLs; verbos HTTP dizem a ação; o status diz o resultado.
- ✓ `@PathVariable` (URL), `@RequestParam` (query string), `@RequestBody` (JSON do corpo).
- ✓ `ResponseEntity` permite escolher o status: `201` no POST, `204` no DELETE.
- ✓ `@Valid` + anotações do DTO validam a entrada; `@RestControllerAdvice` centraliza os erros.
- ✓ Teste com cliente HTTP ou `curl`, inclusive os casos de erro.

Exercícios

1. Crie o CRUD completo de `Livro` (titulo, autor, ano) com as cinco rotas.

2. Faça o `GET /api/livros/{id}` devolver `404` com JSON de erro quando o id não existir.

3. Valide que `ano` não pode ser maior que o ano atual e devolva `400` com mensagem clara.

4. Exporte sua coleção de requisições do cliente HTTP e coloque na pasta `requests/` do projeto.

Para ir além

- Projetos `RestAPI_v1`, `RestAPI_v2`, `RestAPI_v3` e `FullStack/Exemplo_Integracao/Back`
- developer.mozilla.org/pt-BR/docs/Web/HTTP/Status

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 05?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web